RedTail Across Five Vectors: One Campaign Observed on a T-Pot Honeypot, September 2026

Thaddäus Bischoff · Computer engineering student · October 2026

Copyright (c) 2026 Thaddäus Bischoff · CC BY 4.0 · https://github.com/TBjunior/redtail-analysis

1. Executive Summary

Between September 16 and 28, 2026, a single T-Pot honeypot recorded one continuous RedTail campaign on five vectors: 24 SSH sessions, 19 Telnet sessions, 699 HTTP requests, 741 HTTPS requests and 31 requests to the Docker API. The same SSH key fingerprint, staging servers in a single network and identical tooling appear across all of them, so I assess this as one operation (high confidence). The Telnet vector is the most notable observation: none of the public reports I reviewed documents Telnet as a RedTail infection path. The web traffic is a fixed set of 49 URLs per pass aimed only at known weaknesses, and 40 of those 49 requests raised no alert with the ET Open ruleset; 37 of them are GET-only probes for PHPUnit files. The tooling did not change during the window and matches what public reports described between 2024 and 2026. The most useful defenses are basic: no password logins over SSH, no Telnet, no exposed Docker API, no development tools in the web root, and alerting on bursts of 404 responses and on changes to `authorized_keys`. All numbers describe one sensor and are lower bounds, because a 21-hour outage on September 22 left a gap in the data.

2. Background

In May 2024, Akamai described RedTail as cryptomining malware that exploits vulnerabilities to download and execute a script, which then installs a miner and uses the victim's computing resources, and reported that its operators had adopted an exploit for CVE-2024-3400 in Palo Alto Networks PAN-OS [1]. In July 2024, Akamai reported that RedTail had begun exploiting CVE-2024-4577, a PHP-CGI vulnerability, within days of its disclosure in June 2024 [2].

In a SANS Internet Storm Center guest diary from January 2025, Hales described how attackers gained access via SSH using weak credentials and then uploaded RedTail's setup scripts and miner binaries [3]. In November 2025, Candela reported RedTail attacks against exposed Docker Engine APIs on port 2375, using the User-Agent string "libredtail-http" [4]. A later guest diary by Roberts (April 2026) reported web-based attacks that used the same User-Agent string: exploitation attempts against an Apache path traversal and CVE-2024-4577, followed by probing for PHPUnit files [5]. In September 2026, Ng published a dynamic analysis of the x86-64 binary from a RedTail bundle for five architectures, including RISC-V, that had been uploaded to a Cowrie-based sensor [6].

None of the public reports I reviewed (as of September 30, 2026) documents a Telnet-based infection vector for RedTail, whereas this sensor observed one. One diary uses the phrase "SSH/telnet" for the channel of an earlier RedTail observation, without describing a Telnet session [5]. The Docker API queries I recorded confirm [4] rather than add to it. Beyond Telnet, this study contributes a cross-vector view of one campaign: the same key fingerprint and staging servers in one network appear on SSH, Telnet and the web vectors, the tooling stayed identical for the whole window, and most of the web requests raise no alert under the ET Open ruleset.

3. Methodology

Environment

The sensor was a single, privately operated T-Pot HIVE instance (Standard edition), version 24.04.1,

built from the project's master branch at commit 3fefebaf (September 4, 2026). It ran on a cloud-hosted virtual machine that was directly reachable from the internet, processed no production data, and was not connected to any other system of the operator. Local changes are limited to the environment file and the Compose file; the latter includes a larger memory allowance for Logstash and a pinned IP-reputation list, both since September 22, 2026. The pinned list affects only the IP-reputation enrichment, not the recorded events. Honeytrap acted as a TCP catch-all on all ports not served by another honeypot.

The sensor did not scan or probe third parties, and it never contacted the RedTail infrastructure. Its outbound traffic consisted only of automatic reactions of the honeypot software to remote activity: replies, TCP resets, call-backs to a sender's own service port, and one file download ordered by a bot in a Telnet session (not RedTail).

The instance went live on September 15, 2026. The analysis window runs from 00:00 UTC on September 16, 2026, the first full UTC day after go-live, until 19:45 UTC on September 28, 2026, while the campaign continued beyond it.

Data sources

Source	Covers	Role in this study
Cowrie	SSH (22), Telnet (23)	sessions, client fingerprints, uploaded files, command lines
Suricata 8.0.3	all traffic, but not the content of HTTPS	HTTP requests on port 80 and Docker API requests on port 2375 (URL, User-Agent), IDS alerts
h0neytr4p	HTTPS (443, 2087)	HTTPS requests
Honeytrap	every TCP port not served by another honeypot (including 2375)	accepted the Docker API connections; generic listener, does not emulate the Docker API
Tanner/Snare	web application (80)	web honeypot behind port 80 (the counts come from Suricata)

Suricata inspected all traffic as an IDS with the ET Open ruleset (updated daily). It cannot read the content of encrypted HTTPS traffic, so HTTPS requests come from h0neytr4p.

Identifying RedTail traffic

I assigned traffic to RedTail per vector, using features that are specific to that vector. An IP address alone was never enough: infected devices often carry several bots, and one source in this data set ran a different SSH tool alongside its RedTail activity on other vectors (Section 5.3).

- **SSH (Cowrie).** The client banner and HASSH fingerprint of the SSH client, the names of files uploaded in the session, and characteristic command lines (a key file, an SSH client configuration without host-key checking, an SCP call). All 24 sessions come from two fixed source addresses. I included both addresses because their sessions show the bundle upload and loader activity described in Section 5.2. The shared HASSH fingerprint supports this assignment but does not prove it, because many legitimate Go-based tools share it. One of the two addresses is also listed as RedTail in ThreatFox [7].
- **Telnet (Cowrie).** A hex-encoded shell check at the start of the session that prints `auth_ok`, and a hex-encoded success message at the end that decodes to `redtail_bot_telnet_ok`. A feature at the end of a sequence only finds complete attempts: the marker matches 18 sessions, the check at the start matches 19. The 19th session is reported separately in Section 5.2 and assessed less firmly.
- **HTTP, HTTPS and Docker API.** The User-Agent `libredtail-http`. As an additional feature, a staging address in the request payload: on HTTPS in the payload recorded by h0neytr4p, on port 80 in the payload of Suricata's alerts (Suricata's `http` entries do not contain it). The User-Agent can be forged, and the 18 sources seen only on the Docker API are assigned by this feature alone.

Counting

All numbers come from aggregations over a single time window (September 16, 00:00 UTC to September 28, 19:45 UTC), never from chart bar heights or from the sample percentages Kibana shows next to field names.

The unit depends on the vector: sessions for SSH and Telnet, requests for HTTP, HTTPS and the Docker API, and source addresses where I give source counts. I never counted log entries.

4. Limitations

Scope

- One sensor, one time window, and a virtual machine.
- The sensor sees delivery and propagation, not post-compromise behavior. The honeypots simulate and log: no attacker command ran on a real shell, and no captured sample was executed. The one exception by design is the PHP sandbox of the web honeypot (Tanner), which runs injected PHP code in isolation. I did not analyze its output.
- I did not reverse engineer the binaries. The assignment to RedTail rests on file names, the User-Agent, the self-reported marker and the match with public reports [1–6]. Statements about later stages (mining [1], persistence and process masquerading [6], what follows the Docker API query [4]) rely on those reports.

Coverage

- Honeytrap does not emulate the Docker API, so only the first request of that vector is visible. Sources seen only there are assigned by User-Agent alone.
- The payload of requests on port 80 is not part of Suricata's `http` entries.
- The Telnet sessions contain no stored file redirects, unlike the SSH sessions. Very probably Cowrie reads the multi-line command that writes the loader's key file line by line and discards the last line, which holds the redirect, as an unknown command.
- Cowrie accepts almost any login, so success on the first try says nothing about the attacker's knowledge.

Gaps in the data

- On September 22 an operational outage of about 21 hours (01:43 to about 23:08 UTC) left a gap in the indexed data for all sensors: indexing was down, and the server was also switched off temporarily for repairs.
- A nightly maintenance reboot interrupts recording for a few minutes around 01:43 UTC. Individual sensors also show occasional empty hours (Tanner and h0neytr4p at low traffic, at most three per day). A day with a zero count therefore means "not observed", not "not attacked", and a missed pass cannot be fully excluded for HTTP on September 21 and 23 and for HTTPS on September 17.

Detection statements and attribution

- IDS statements apply only to the ET Open ruleset. In the examples I checked, Suricata alerted on Docker API and HTTPS traffic only for the source address's reputation (and, on the Docker API port, a TCP stream anomaly), not for the request.
- The 19th Telnet session is assigned less firmly than the 18 with the marker: it matches the opening sequence but not the marker, and its source shows other RedTail activity on two further vectors.

5. Observations

This chapter describes only what was observed. Interpretation follows in Section 6.

5.1 Timeline

All times are UTC. Sessions (SSH, Telnet) and requests (HTTP, HTTPS, Docker API) are counted per day, never log entries.

Activities per day:

Date (UTC)	SSH sessions	Telnet sessions	HTTP (80) requests	HTTPS requests	Docker API requests
2026-09-16	2	1	52	48	1
2026-09-17	1	1	49	0	2
2026-09-18	0	2	49	50	0
2026-09-19	4	2	98	110	5
2026-09-20	0	1	98	50	3
2026-09-21	6	3	0	96	4
2026-09-22[1]	0	1	0	0	0
2026-09-23	0	1	0	48	2
2026-09-24	6	3	146	97	1
2026-09-25	0	1	12	144	3
2026-09-26	1	3	97	0	1
2026-09-27	4	0	49	97	3
2026-09-28[2]	0	0	49	1	6
Total	**24**	**19**	**699**	**741**	**31**

[1] No data was indexed from 01:43 until about 23:08 UTC (21 hours 25 minutes), for all sensors; hourly checks show no entries between 02:00 and 22:00 UTC. This is an operational outage, not attacker behavior (Section 4).

[2] The analysis window ends at 19:45 UTC.

First and last connection per vector:

Vector	First connection (UTC)	Last connection (UTC)
Docker API	2026-09-16 01:38[3]	2026-09-28 16:58[3]
SSH	2026-09-16 04:04	2026-09-27 05:29
Telnet	2026-09-16 06:39	2026-09-26 15:42
HTTPS	2026-09-16 18:06	2026-09-28 05:50
HTTP (80)	2026-09-16 22:44	2026-09-28 08:09

[3] Suricata time, about one minute after the actual request (flow timeout).

Selected events:

Date and time (UTC)	Event
2026-09-17	SSH session whose command lines contain the loader's key file; its key fingerprint reappears on September 20.
2026-09-18 08:50	Telnet session that breaks off after the shell check, without the marker (the 19th session).
2026-09-19 06:28	SSH session of 2.9 seconds: one login, one long command line, SCP upload of the complete bundle (seven files).
2026-09-20	One HTTP connection stream of 16 seconds contains a complete pass of 49 requests: 42 GET requests and seven POST exploitation attempts (five PHP-CGI argument injection, two Apache path traversal); the payload carries the same key fingerprint as the SSH session of September 17, from a different source address.
2026-09-	Data gap, all sensors (footnote 1).

22 01:43 to about 23:08	
2026-09-24	Last of eight SCP upload sessions in the window; all seven bundle files unchanged since September 16.

All five vectors were active from the first day: the first contacts on September 16 follow each other within about 21 hours, from the Docker API at 01:38 to HTTP at 22:44. Telnet sessions and Docker API requests each occur on 11 of the 13 days, SSH sessions on seven days, and web passes arrive in bursts, for example 146 HTTP requests on September 24 and 144 HTTPS requests on September 25. Every day of the window shows activity on at least one vector, including the day of the outage.

5.2 Infection vectors

Vector	Captured by	Unit	Count	Identifying features
SSH (22)	Cowrie	sessions	24 (2 sources)	client `SSH-2.0-Go`, HASSH `16443846184eafde36765c9bab2f4397`; bundle upload by SCP in 8 sessions, loader in 6
Telnet (23)	Cowrie	sessions	19 (19 sources, one session each)	18 complete sessions with marker, 1 aborted; 34 log entries per complete session
HTTP (80)	Suricata (`http`)	requests	699 (19 sources)	fixed set of 49 URLs per pass; User-Agent `libredtail-http`; staging address in the payload of the alerts of all 19 sources
HTTPS (443)	h0neytr4p	requests	741 (22 sources)	set of 48 requests per pass; User-Agent `libredtail-http`; staging address in the payload of all 22 sources
Docker API (2375)	Suricata (`http`), Honeytrap	requests	31 (28 sources)	single request for the container list; User-Agent `libredtail-http`

SSH. Two fixed source addresses opened 24 sessions with the same Go-based client. Each session makes exactly one login attempt with a default password (for example `root` with `admin`). The sensor accepted 17 and rejected 7. Eight sessions upload a bundle of seven files by SCP, five architecture-specific binaries (`arm7`, `arm8`, `i686`, `riscv`, `x86_64`) and two shell scripts; six sessions use the loader. The session recorded on September 19 lasted 2.9 seconds from connection to close.

Telnet. Nineteen sources each opened exactly one session. In the 18 complete sessions the log has exactly 34 entries: 17 command lines plus negotiation, login and end. Eight of the command lines are unknown to Cowrie: three attempts to escape from the restricted shell of a network device and five lines of the multi-line command that writes the loader's key file. Every session starts with an identity query, a read of the local account file and the hex-encoded shell check; the 18 complete sessions end with the hex-encoded success message. Only the loader is used; no file is uploaded. The 19th session (September 18, 08:50) breaks off after the shell check, without the marker.

HTTP (80). Each pass requests the same 49 distinct URLs, one per target in the table below, including the Docker API's container-list path as an ordinary URL. The only User-Agent is `libredtail-http`. As a cross-check, the 14 container-list requests inside the port-80 passes and the 31 requests on the Docker API port add up to 45, which matches an independent count by URL. Suricata's alerts on these requests carry the request payload: 603 alerts from 18 sources name the first staging address and 22 alerts from one source name the second (Section 5.3), which accounts for all 19 sources.

HTTPS (443). h0neytr4p recorded 741 requests from 22 sources, again in passes of 48 requests with one request for the container-list path. A staging address appears in the payloads of all 22 sources: the first for 20 of them, the second for two (Section 5.3).

Docker API (2375). Twenty-eight sources sent 31 requests for the container list to the standard Docker API port. Honeytrap accepted the connections and did not answer, because it does not

emulate the Docker API, so only the first step of this vector is visible. Eighteen sources did nothing else. Ten also showed web activity on port 80 or 443. Other scanners also probe this port (for example with a Go HTTP client or zgrab), so only requests with the RedTail User-Agent were counted.

Composition of one pass on port 80:

Target	URLs per pass	Request type	IDS alert (ET Open)
PHPUnit `eval-stdin.php` (CVE-2017-9841) [8]	37	probing for the file associated with the CVE: GET only, 37 possible locations, no exploitation attempt	none
PHP-CGI argument injection (CVE-2024-4577) [9]	5	exploitation attempts targeting the CVE, five spellings	yes, with CVE
Apache path traversal (CVE-2021-41773, CVE-2021-42013) [10, 11]	2	exploitation attempts targeting the CVEs, single and double encoded; CVE-2021-42013 bypasses the incomplete fix for CVE-2021-41773	yes, with CVE
ThinkPHP 5.x, `invokefunction` route (likely CVE-2019-9082) [12]	2	test whether code execution is possible	yes, but the rule names no CVE
PHP file inclusion via `pearcmd`	2	technique that requires a misconfiguration, no CVE [1]	none
Docker API path (`/containers/json`)	1	probe for a misconfiguration, no CVE	none
Total	**49**		

For PHP-CGI and Apache the CVE numbers come from the rule metadata of the Suricata alerts. The ThinkPHP rule carries none. I assign CVE-2019-9082 because its NVD description names exactly this route, although the NVD version range ("before 3.2.4") is imprecise and the route is the known ThinkPHP 5.x flaw. Akamai lists CVE-2018-20062 for RedTail, a different ThinkPHP route [1]. The PHPUnit entry is assigned to its CVE by the file path alone, and the PHPUnit paths were requested with GET only: 521 GET and no POST requests in the window. Akamai found PHPUnit exploit paths and CGI-bin RCE paths among the strings embedded in the 2024 binary and did not see them exploited in that campaign [1]. Roberts reports probing for PHPUnit files in March 2026 [5]. CVE-2024-4577 bypasses the protection added for the original PHP-CGI weakness CVE-2012-1823 [13]. Of the 56 web-facing sources (ports 80, 443 and 2375), 38 carry a staging address in a payload (22 on HTTPS, 19 on port 80 through alert payloads, three on both). The other 18 appear only on the Docker API port and are assigned by User-Agent alone.

5.3 Linking the vectors

The following features are shared across vectors:

- **Key fingerprint.** The same ED25519 key fingerprint (comment `dlr@sftp`) appears in the loader's commands on SSH, Telnet and in web payloads. It is proven identical between an SSH session on September 17 and an HTTP request on September 20, from different source addresses.
- **Staging addresses.** The first address, `217[.]60[.]103[.]56`, appears in the loader's commands on SSH and Telnet and in web payloads: 20 HTTPS sources, and 18 port-80 sources through the payloads of Suricata alerts. A second address, `217[.]60[.]195[.]113`, appears in the payloads of two HTTPS sources (September 20 and 25) and one port-80 source. The second address is listed in the public ThreatFox database as RedTail infrastructure since June 10, 2026, described there as an SCP distribution host for the user `dlr` [14].
- **Files.** The seven bundle files have unchanged SHA-256 hashes across all upload sessions (September 16 to 24). The two helper files of the loader are identical in six sessions between September 17 and 27.
- **Client and session structure.** All 24 SSH sessions share one HASSH fingerprint, and all complete Telnet sessions have the same structure (34 entries).

- **Shell check.** The same hex-encoded `auth_ok` check appears in the long SSH command and at the start of every Telnet session.
- **Campaign tag.** The loader calls its staged script with a tag that names the entry route (for example `ssh`, `telnet`, `apache`, `cve_2024_4577`).
- **Sources on several vectors.** Five sources appear on more than one vector, two of them on three. An address alone does not link them: one of these sources also connected five times over SSH with a different client and HASSH fingerprint (four failed logins, one accepted login as `root`, closed immediately, no command). Sources are therefore linked through the features of each vector, and the address only connects what has already been assigned.

5.4 Tooling stability

The following stayed unchanged during the window:

- **Bundle.** The seven bundle files kept the same SHA-256 hashes across eight upload sessions between September 16 and 24.
- **Loader helper files.** Two files, identical in six sessions between September 17 and 27.
- **SSH client.** One HASSH fingerprint in all 24 sessions.
- **Telnet sessions.** Exactly 34 entries with the same structure in each of the 18 complete sessions.
- **Web request sets.** 49 URLs per pass on port 80 and 48 requests per pass on HTTPS.

6. Analysis and Assessment

Each assessment states its confidence (high, moderate or low) and the basis for it. The underlying observations are in Section 5.

6.1 One operation behind all vectors (high confidence). The same key fingerprint and staging addresses from one network appear on SSH, Telnet and in web payloads. The bundle files and the loader's helper files are identical across the window, all SSH sessions share one client fingerprint, all complete Telnet sessions share one structure, and the same hex-encoded shell check appears in the long SSH command and at the start of every Telnet session. These are properties of the tooling, not of an IP address (Section 5.3). Addresses in one network show co-location, not necessarily one operator. The assessment rests on their combination with the key fingerprint and the tooling. Structural similarity alone would not prove one operator, because Hales reports the same two scripts in four attacks from three source addresses, with different binaries between them [3]. Here the identical file hashes within the window and the shared key fingerprint carry the assessment. Caveat: "RedTail" is the name that the file names, the User-Agent and the marker give this tooling and that public reports [1–6] use. I did not reverse engineer the binaries.

6.2 Two-stage distribution (moderate confidence). The full bundle arrives over SSH from only two fixed addresses. Telnet and web traffic comes from many sources (19 on Telnet, 56 on the web vectors) and uses only the lean loader, which labels itself `bot` in its Telnet success message. Five sources appear on several vectors. This fits a small number of delivery servers plus a large population of already infected devices that scan on their own. Caveats: I have not verified that the Telnet and web sources are infected devices. Source distribution alone cannot separate a worm from an operator: a wave of the publicly known `mdrfckr` key family on September 19 and 20 came from about fifty scattered, non-data-center sources, which I read as compromised machines searching on their own. In the RedTail data the distribution does separate the delivery stage from the spreading stage. One of the two fixed SSH sources was already listed as RedTail infrastructure in ThreatFox in April 2026 [7], which fits long-lived delivery servers.

6.3 Broad, automated delivery with a capable toolkit (high confidence for broad and automated, moderate for capable). Entry uses default credentials with a single attempt per session. The web requests are fixed sets that never adapt to the responses. The PHP-CGI requests rely on a character-set conversion that works on Windows, yet the sensor, a Linux host, received

them anyway; Akamai drew the same conclusion in 2024 from a script that covered architectures irrelevant to Windows [2]. Against this, the bundle covers five architectures including RISC-V, still a rare target: the January 2025 bundle in [3] has four binaries and no RISC-V build, while a report from September 2026 [6] and my data show five. This suggests active maintenance, and the loader has two fetch paths (6.4).

6.4 A loader built for robustness and for taking over the account (moderate confidence). The loader fetches its next stage by SCP and falls back to HTTPS with certificate checks switched off, which suggests the operator expects outbound SSH to be filtered. The shell check's success message is hex-encoded. Hales reads it as the confirmation that the key was written [3]. A side effect is that it does not appear in plain-text log searches, and I cannot tell whether that is intended. For `authorized_keys`, the commands first remove protective file attributes, then overwrite the file instead of appending to it, then set the immutable flag. If run on a real host, this would remove every existing key and keep the administrator from changing the file. What I observed are the command lines. I did not observe their effect. A further reading is less certain (low confidence): the staged script receives a tag naming the entry route, which could let the operator measure which route pays off. I have no evidence of what the operator does with the tag. The tag is long-standing: `cve_2024_4577` already appears in Akamai's 2024 sample [2], and the `setup.sh` of the SSH bundle starts the binary with the argument `ssh` [6].

6.5 The web reconnaissance is largely invisible to a standard IDS ruleset (high confidence for ET Open). 40 of the 49 requests in a pass raised no alert: the 37 PHPUnit paths, the two `pearcmd` requests and the Docker API path. Only the nine exploitation attempts (five PHP-CGI, two Apache, two ThinkPHP) triggered alerts in the request direction. The PHPUnit paths are only requested with GET, so rules that look for exploit code in a POST body never match. Suricata cannot read HTTPS; and in the examples I checked, the alerts on the Docker API port and on HTTPS concern the source address's reputation (and a TCP stream anomaly), not the request. Reputation lists change daily. A useful early-warning sign in this data is a burst of 404 responses for development-tool paths such as `vendor/phpunit`. It is plausible that the actual attack follows only after a positive answer, which the honeypot never gave, but I cannot confirm this (low confidence). Other rulesets or products may behave differently.

6.6 Comparison with public reporting (moderate confidence). Where my data matches public reports, it supports the identification as RedTail, where it differs, it shows how the tooling has changed.

- *Matches.* The SSH delivery follows the pattern Hales observed in October 2024, published January 2025: an upload of setup and clean scripts and architecture-specific binaries, then a fixed sequence that removes the protective attributes from `authorized_keys`, writes a key with the comment `rsa-key-20230629`, sets the attributes again and prints the hex-encoded check [3]. The same comment appears in my data. A report from September 2026 describes a bundle for five architectures including RISC-V [6]. The web sets match Roberts's description of POST requests for Apache path traversal and CVE-2024-4577 followed by probing for PHPUnit files [5]; the soft-hyphen PHP-CGI request and the loader tag `cve_2024_4577` also appear in a RedTail sample that Akamai documented within days of the CVE's disclosure [2]. The Docker API request carries the User-Agent and port that Candela reported [4]; what follows the first request I did not observe. One SSH source and the second staging address are listed as RedTail infrastructure in ThreatFox [7, 14].
- *Differences.* The 2024 bundle in [3] has no RISC-V build. Roberts describes two PHP-CGI requests per pass [5], my sets contain five spellings. The tags in [4] and [5] carry the suffix `selfrep`, mine do not. In 2024, Akamai described the delivery infrastructure as several unrelated servers at different hosting companies [1]. The two staging addresses in my data lie in one network.
- *Telnet.* The Telnet sessions carry the same key, staging address and marker as the other vectors, so I am confident they belong to the same tooling. None of the public reports I reviewed documents Telnet as an infection path. That means only that I did not find it documented; it does

not show that nobody else has seen it.

6.7 Several staging addresses in one network, in parallel and over time (moderate confidence). Two staging addresses were in use during the window, one of them by only a few sources (one on port 80, two on HTTPS). The both lie in the same network. The second address was already listed in June 2026 [14], and the address `178[.]16[.]55[.]224` appears in reports from November 2025 [4] and March 2026 [5], so addresses are used in parallel and reused over months, not only replaced.

7. MITRE ATT&CK Mapping

The mapping is my reading of the command lines and requests recorded on the sensor. The honeypots simulate and log, so "observed" means that the behavior was attempted and recorded, not that it had an effect on a real host.

Technique	ID	Evidence from own data	Observed / reported
Active Scanning: Vulnerability Scanning	T1595.002	Fixed web request set of 49 URLs per pass; the 37 PHPUnit paths are requested with GET only (probing for the file)	Observed
Exploit Public-Facing Application	T1190	Exploitation attempts targeting CVE-2024-4577 (five spellings), CVE-2021-41773/42013 and the ThinkPHP route of CVE-2019-9082; recorded as requests, the honeypot is not vulnerable	Observed (attempts)
Valid Accounts: Default Accounts	T1078.001	SSH sessions with one login attempt each and default passwords; 17 accepted and 7 rejected by the honeypot	Observed
Container and Resource Discovery	T1613	31 requests for the container list on the Docker API port from 28 sources	Observed (first request only); follow-up reported [4]
Command and Scripting Interpreter: Unix Shell	T1059.004	Shell command lines in SSH and Telnet sessions; shell scripts in the bundle	Observed (command lines, not executed)
System Information Discovery	T1082	Kernel and architecture query in the SSH command, used to start the matching binary	Observed
Account Discovery: Local Account; System Owner/User Discovery	T1087.001; T1033	Telnet sessions open with an identity query and a read of the local account file	Observed
Account Manipulation: SSH Authorized Keys	T1098.004	`authorized_keys` overwritten with the operator's public key (comment `rsa-key-20230629`)	Observed (command line)
File and Directory Permissions Modification: Linux and Mac	T1222.002	Immutable and append-only attributes removed from `authorized_keys` first, set again afterwards	Observed (command line)
Obfuscated Files or Information	T1027	Hex-encoded shell check and success message; base64-encoded payload in the body of the PHP-CGI requests	Observed
Ingress Tool Transfer	T1105	Next stage fetched from the staging server by SCP with an HTTPS fallback; bundle uploaded by SCP in 8 sessions	Observed (command lines, not executed)
Indicator Removal: File Deletion	T1070.004	Setup scripts, key file and client configuration deleted after use	Observed (command lines)

8. Indicators of Compromise

All values were observed between September 16 and 28, 2026 (UTC). Network indicators age quickly. File hashes are stable within the window but differ from those published in [3] and [6]. The key comment `rsa-key-20230629` has been reported since observed in October 2024 [3]. Values are

defanged. The list contains only infrastructure of the campaign operator, no victim addresses, no keys and no samples.

Type	Value	Context
IPv4	`217[.]60[.]103[.]56`	First staging address, named in the loader's commands: SCP login as user `dlr`, HTTPS fallback `hxxps://217[.]60[.]103[.]56/sh`. In the payloads of 20 HTTPS sources and, through Suricata alerts, of 18 port-80 sources. Never contacted from the sensor.
IPv4	`217[.]60[.]195[.]113`	Second staging address, same network. In the payloads of two HTTPS sources (September 20 and 25) and of one port-80 source (22 alerts). Listed in ThreatFox since June 10, 2026 as an SCP distribution host for user `dlr` [14]. Never contacted from the sensor.
IPv4	`130[.]12[.]180[.]51`	Fixed SSH source: bundle upload and loader sessions. Listed as RedTail in ThreatFox since April 9, 2026 [7].
IPv4	`77[.]90[.]185[.]20`	Second fixed SSH source.
SSH key fingerprint	`SHA256:O/at8341SoPpKvTPvMsJSgjQm30md9VTS2it25sY0vg`	ED25519 key (comment `dlr@sftp`) with which the loader logs in to the staging server. Identical across vectors within the window; I did not find it in the public reports I read. The key itself is not reproduced.
`authorized_keys` comment	`rsa-key-20230629`	Comment of the public key written to `authorized_keys` in the bundle session; matches the default naming of PuTTYgen (`rsa-key-YYYYMMDD`). The same comment is reported in [3] (January 2025).
SSH client	banner `SSH-2.0-Go`, HASSH `16443846184eafde36765c9bab2f4397`	Client of all 24 SSH sessions. Shared by many legitimate Go tools; use only in combination with other indicators.
User-Agent	`libredtail-http`	HTTP, HTTPS and Docker API requests. Can be forged.
Telnet marker	`redtail_bot_telnet_ok`	Hex-encoded success message at the end of complete Telnet sessions; invisible to plain-text log searches.
File names	`redtail.arm7`, `redtail.arm8`, `redtail.i686`, `redtail.riscv`, `redtail.x86_64`, `setup.sh`, `clean.sh`	The seven bundle files uploaded by SCP.
Loader artifacts	`key.ppk`, `sshcfg`	Temporary files the loader creates in `/tmp`, `/var/tmp` or `/dev/shm` and deletes afterwards.
SHA-256	`1e70b63472772e3f5092ffe9c3573470e73590e6ab6d93fdcede1d368a5fd72d`	`setup.sh`
SHA-256	`3f3a11bafabb1a35db913cfe51995f2e357d049e268860175876ae5a93d23892`	`clean.sh`
SHA-256	`3f3bf218089d1488617d37f8a5116bb2791eb39ce06a1b5bc9a4cdfe5e94dd39`	`redtail.riscv`
SHA-256	`8e1a67a5c03b3cd818f046c7a1605afccc0ee5ce437a0d099881f1872b54bc70`	`redtail.i686`
SHA-256	`d1cac82f44b54b0fd244a9e4122811e9ae108a197c7a65a20fd2e7552683e68e`	`redtail.arm8`
SHA-256	`d70f917e35813a7ae323e6b2b539d6dbbfc3a3a6599f1fed93430b14ca08b141`	`redtail.arm7`
SHA-256	`f0aa83bbbd2c75e2f71ec16029ee5fcfad59f3a8efa30a500b815f0f6c18d987`	`redtail.x86_64`

SHA-256	`0db4656687a425c47d19000db866db52c7e415dbfaf6b5c651adcb9275ab23ca`	`key.ppk`, the loader's SSH key file (OpenSSH private key format, 405 bytes; the key is not reproduced)
SHA-256	`ae8d459595257f2f22c9d1ff74c4fb8a91643fad7899b57556496716692b904e`	`sshcfg`, the loader's SSH client configuration (55 bytes, text)

9. Recommendations for Defenders

Each recommendation follows from an observation in this study. I have not validated them on a production system, and the thresholds are examples, not tested values.

Area	Finding in this study	Recommendation
Prevent	All SSH sessions used default passwords, with one attempt per connection.	Allow SSH logins by key only, with one attempt per connection, rate limiting alone does not help.
Prevent	A whole delivery path ran over Telnet (19 sessions from 19 sources).	Disable Telnet on routers, IoT devices and servers, change default passwords and update firmware.
Prevent	The Docker API was queried on its standard port (31 requests from 28 sources).	Never expose the Docker API: use the local socket or an SSH context for remote access, block ports 2375 and 2376 in the firewall or security group, and use TLS with client certificates where remote TCP access is unavoidable.
Prevent	Most of the web set searches for PHPUnit (37 of 49 URLs per pass).	Keep development tools out of production: `composer install --no-dev`, and keep `vendor/` outside the web root.
Prevent	All targeted weaknesses had been known for months to years.	Patch them: PHP 8.1.29, 8.2.20 or 8.3.8 or later, and PHP-FPM instead of PHP-CGI (CVE-2024-4577, mainly PHP under Windows, for example XAMPP); Apache 2.4.51 or later, and `mod_cgi` off if unused (CVE-2021-41773/42013); a maintained ThinkPHP version (CVE-2019-9082; the `invokefunction` flaw is fixed in 5.0.23 and 5.1.31).
Detect	40 of 49 web requests per pass raised no alert.	Alert on bursts of 404 responses for development-tool paths such as `vendor/phpunit`, for example a ban after three such requests within 60 seconds.
Detect	`libredtail-http` is the only User-Agent in the web set.	Alert on or block it as a supplement only
Detect	All SSH sessions share one Go client fingerprint (HASSH).	Use HASSH only in combination with other indicators; many legitimate Go tools share it.
Detect	The loader replaces `authorized_keys` and sets the immutable flag.	Monitor `authorized_keys` and its attributes, for example with an auditd watch on `.ssh` directories, and check `lsattr` for the immutable flag.
Detect	The loader fetches its next stage by SCP or HTTPS.	Watch outbound `scp`, `wget` and `curl` from web or service users, and restrict their outbound SSH. Roberts also suggests alerting on requests to a bare IP address with the path `/sh`, which is rarely seen in legitimate traffic [5].
Detect	Public reports describe RedTail as a cryptominer [1, 3].	Watch for mining symptoms: sustained high CPU load, unknown processes, new cron jobs, and programs in `/tmp`, `/var/tmp` or `/dev/shm`; mount temporary directories `noexec` where the software allows it.
Respond	The loader sets the	After a suspected compromise, check `authorized_keys` with its attributes, cron jobs

```
        immutable flag on a  and temporary directories; rebuild the system instead of cleaning it.
        uthorized_keys.
```

Blocking the staging addresses outbound is useful but ages quickly.

10. Conclusion

One T-Pot sensor recorded a single RedTail operation on five vectors between September 16 and 28, 2026, with the same key fingerprint, staging servers in one network and tooling throughout. The SSH delivery follows the pattern public reports described in January 2025, with a RISC-V build added [3, 6]. The observation adds Telnet to the delivery paths described in public reporting and shows that most of the campaign's web reconnaissance raises no alert under the ET Open ruleset. For defenders the lessons are old ones: close the doors the toolkit tries (password logins over SSH, Telnet, an exposed Docker API, development tools in the web root, known PHP and Apache flaws), and watch for 404 bursts and changes to `authorized_keys`. The findings describe one sensor in one window and say nothing about how widespread RedTail is.

Kurzfassung (Deutsch)

Ein einzelner T-Pot-Honeypot hat vom 16. bis 28. September 2026 (UTC) eine zusammenhängende RedTail-Kampagne über fünf Wege aufgezeichnet: 24 SSH-Sitzungen, 19 Telnet-Sitzungen, 699 HTTP-Anfragen, 741 HTTPS-Anfragen und 31 Anfragen an die Docker-API. Derselbe SSH-Schlüssel-Fingerabdruck, Staging-Server in einem einzigen Netz und identische Werkzeuge treten auf allen Angriffswegen auf. Ich bewerte das als eine einzige Operation (hohe Sicherheit). Telnet als Infektionsweg ist in den von mir gesichteten öffentlichen Berichten nicht dokumentiert. Von den 49 Web-Anfragen eines Durchlaufs lösten 40 mit dem Regelsatz ET Open keinen Alarm aus. Die Kampagne lief nach dem Analysefenster weiter. Verteidiger erreichen am meisten mit Grundlagen: keine Passwort-Anmeldung per SSH, kein Telnet, keine offene Docker-API, keine Entwicklungswerkzeuge im Web-Root, dazu Alarme bei gehäuften 404-Antworten und bei Änderungen an `authorized_keys`. Alle Zahlen beschreiben einen Sensor und sind Untergrenzen, denn am 22. September fehlen rund 21 Stunden Daten.

References

1. Akamai SIRT: *RedTail Cryptominer Threat Actors Adopt PAN-OS CVE-2024-3400 Exploit*, 2024-05-30. https://www.akamai.com/blog/security-research/2024-redtail-cryptominer-pan-os-cve-exploit
2. Akamai SIRT: *CVE-2024-4577 Exploits in the Wild One Day After Disclosure*, 2024-07-10. https://www.akamai.com/blog/security-research/2024-php-exploit-cve-one-day-after-disclosure
3. C. Hales: *Examining Redtail Analyzing a Sophisticated Cryptomining Malware and its Advanced Tactics [Guest Diary]*, SANS ISC, 2025-01-09. https://isc.sans.edu/diary/31568
4. M. Candela: *RedTail Cryptominer: First Evidence of Docker API Targeting*, Beelzebub, 2025-11. https://beelzebub.ai/blog/redtail-cryptominer-first-evidence-of-docker-api-targeting/ (also published at https://itnext.io/redtail-cryptominer-first-evidence-of-docker-api-targeting-c061096443f8, 2025-11-14)
5. J. Roberts: *Danger of Libredtail [Guest Diary]*, SANS ISC, 2026-04-29. https://isc.sans.edu/diary/32936
6. A. Ng: *Redtail Payload Analysis [Guest Diary]*, SANS ISC, 2026-09-09. https://isc.sans.edu/diary/33326
7. abuse.ch ThreatFox: IOC 1783010 (`130[.]12[.]180[.]51:22`, malware RedTail, first seen 2026-04-09). https://threatfox.abuse.ch/ioc/1783010
8. NIST NVD: *CVE-2017-9841* (PHPUnit `eval-stdin.php`). https://nvd.nist.gov/vuln/detail/CVE-2017-9

841
9. NIST NVD: *CVE-2024-4577* (PHP-CGI argument injection). https://nvd.nist.gov/vuln/detail/CVE-2024-4577
10. NIST NVD: *CVE-2021-41773* (Apache HTTP Server 2.4.49, path traversal). https://nvd.nist.gov/vuln/detail/CVE-2021-41773
11. NIST NVD: *CVE-2021-42013* (Apache HTTP Server 2.4.50, incomplete fix for CVE-2021-41773). https://nvd.nist.gov/vuln/detail/CVE-2021-42013
12. NIST NVD: *CVE-2019-9082* (ThinkPHP, `invokefunction` route). https://nvd.nist.gov/vuln/detail/CVE-2019-9082
13. NIST NVD: *CVE-2012-1823* (PHP-CGI, original weakness). https://nvd.nist.gov/vuln/detail/CVE-2012-1823
14. abuse.ch ThreatFox: IOC 1825826 (`217[.]60[.]195[.]113:22`, malware RedTail, first seen 2026-06-10). https://threatfox.abuse.ch/ioc/1825826/

Use of AI

Claude (Anthropic) was utilized for assistance with refining grammar, enhancing the professionalism and readability of this analysis, and with reviewing of this document. All research, analysis, and technical details are my own or are referenced above.

This is a personal research project. It is not affiliated with or endorsed by my university or employer.
